

recursive-dedup-audit

Recursive Submodule Deduplication Audit (post-Phase 1.5)

Date: 2026-05-06

Scope: All 6 reachable `.gitmodules` in the meta-repo tree (excluding third-party trees: `cli_agents/`, `cli_agents_resources/`, `dependencies/HuggingFace_Hub`, `dependencies/Ollama`, `dependencies/LLama_CPP`).

Inventory

- Total recursive submodules: **241**
- `.gitmodules` files inspected: 6
 - `./`.gitmodules (meta-repo root)
 - `./challenges/`.gitmodules
 - `./helix_agent/`.gitmodules
 - `./helix_agent/HelixLLM/`.gitmodules
 - `./helix_qa/`.gitmodules
 - `./helix_qa/tools/opensource/skyvern/`.gitmodules
- (URL, path) entries collected: **244**
- URLs appearing at >1 path (potential duplicates): **27**

Categorization of the 27 multi-path URLs

Category A — IN-TREE third-party legitimately at two locations (1)

URL	Paths	Disposition
<code>git@github.com:jeremylongshore/claude-code-plugins-plus-skills.git</code>	<code>cli_agents/</code> <code>bridle,</code> <code>cli_agents/</code> <code>claude-</code> <code>plugins</code>	SKIP — both paths are inside <code>cli_agents/</code> (third-party), excluded from removal per task constraints.

Category B — Orphan `.gitmodules` entries with NO tracked gitlink (3)

These are stale `[submodule]` blocks in `helix_agent/.gitmodules`. The corresponding directories are tracked as plain files (not gitlinks), or are not tracked at all. Safe to remove.

Name	Path	Notes
<code>Toolkit/</code> <code>SiliconFlow</code>	<code>helix_agent/</code> <code>Toolkit/</code> <code>SiliconFlow</code>	<code>helix_agent/Toolkit/</code> is checked in as plain files. No gitlink.
<code>Toolkit/Chutes</code>	<code>helix_agent/</code> <code>Toolkit/Chutes</code>	Same. URL also has duplicate (next row).

Name	Path	Notes
Toolkit/Toolkit/ Chutes	helix_agent/ Toolkit/Toolkit/ Chutes	Pathological double-nest. Path doesn't exist on disk.

Category C — Vendored dependency layout via go.mod replace (23) — DO NOT REMOVE

These look like duplicates by URL but are **legitimate vendored dependency trees** required for compilation:

C1: helix_agent/HelixLLM/submodules/* (22 entries)

Wired into helix_agent/HelixLLM/go.mod via replace directives. Removing them breaks HelixLLM's build:

```
replace digital.vasic.agentic      => ./submodules/Agentic
replace digital.vasic.auth         => ./submodules/Auth
replace digital.vasic.background   => ./submodules/BackgroundTasks
replace digital.vasic.cache        => ./submodules/Cache
replace digital.vasic.challenges   => ./submodules/Challenges
replace digital.vasic.concurrency  => ./submodules/Concurrency
replace digital.vasic.database     => ./submodules/Database
replace digital.vasic.embeddings   => ./submodules/Embeddings
replace digital.vasic.eventbus     => ./submodules/EventBus
replace digital.vasic.formatters   => ./submodules/Formatters
replace digital.vasic.helixqa      => ./submodules/HelixQA
replace digital.vasic.llmorchestrator => ./submodules/LLMOrchestrator
replace digital.vasic.llmprovider  => ./submodules/LLMProvider
replace digital.vasic.mcp          => ./submodules/MCP_Module
replace digital.vasic.memory       => ./submodules/Memory
replace digital.vasic.observability => ./submodules/Observability
replace digital.vasic.optimization => ./submodules/Optimization
replace digital.vasic.planning     => ./submodules/Planning
replace digital.vasic.rag          => ./submodules/RAG
replace digital.vasic.streaming    => ./submodules/Streaming
replace digital.vasic.toolschema   => ./submodules/ToolSchema
replace digital.vasic.vectordb     => ./submodules/VectorDB
replace digital.vasic.conversation => ./submodules/conversation
```

C2: helix_agent/<Name> paired with root or helix_agent/Toolkit (1)

helix_agent/Challenges (160000 gitlink) is wired into helix_agent/go.mod:

```
replace digital.vasic.challenges => ./Challenges
```

It is duplicated by URL with the root challenges/ submodule, but removal would break HelixAgent's build. The two paths represent **different dependency-resolution roots**.

Architectural decision required (NOT performed)

Promoting `helix_agent/HelixLLM` to consume `../../../../Containers,../../../../Security,../../../../helix_agent/Auth`, etc. via root canonicals requires: 1. Rewriting all `replace` paths in `HelixLLM's go.mod`. 2. Verifying `go mod tidy` + full `HelixLLM` build still pass. 3. Same for `helix_agent/go.mod`. 4. Confirming with the user that **submodule reuse via parent-traversal `replace` paths** is the desired architecture (vs. each module having its own vendored dep tree, which is the current — and arguably intentional — state).

This is **out of scope for a mechanical dedup pass** and demands a Phase 2 architectural ticket.

What was removed (this WP)

Only the 3 orphan `Toolkit/*` entries from `helix_agent/.gitmodules`. They had no corresponding tracked gitlinks, so removal is purely a config cleanup with zero functional impact.

Possibly-unused submodules (REPORT ONLY — NOT removed)

A `grep`-based “is anything referencing this submodule?” pass would be misleading because `Go replace` directives reach via `go.mod`, which is itself a reference. Determining “unused” requires:

- 1. `go mod tidy` per module to detect unused `replace` targets.
- 2. Build + test each module to confirm.
- 3. Walk Challenge scripts that may exercise submodules outside the Go build.

This is deferred to a dedicated WP. The current state is intentionally conservative: nothing removed without proof of redundancy.

Summary

Bucket	Count	Action
Total URL duplicates flagged	27	—
Truly safe to remove	3 (orphan <code>.gitmodules</code> entries)	Removed
Vendored dependency layout (legitimate)	23	Preserved; needs architectural decision
Third-party (out of scope)	1	Skipped

No nested submodules were physically removed. The audit revealed that the remaining “duplicates” are intentional vendored dependency trees, not redundant copies. Phase 1.5 WP3 already handled the simpler cases.